

Python Programming

Unit 02 – Lecture 05 Notes

Advanced Functions and Functional Patterns

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	2
2	Types of Functions in Python (and When to Use Them)	2
2.1	Built-in Functions (Provided by Python)	2
2.2	User-Defined Functions (<code>def</code>)	3
2.3	Functions by Arguments and Return Value (Common academic view)	3
2.4	Function Parameter Styles (Practical Python)	3
2.5	Anonymous Functions (<code>lambda</code>)	4
2.6	Higher-Order Functions (Functions that take/return functions)	4
2.7	Predicate Functions (Return <code>True/False</code>)	5
2.8	Key Functions (Used for Sorting/Ranking)	5
2.9	Recursive Functions (Define a problem in terms of itself)	5
2.10	Generator Functions (<code>yield</code>) (Lazy computation)	5
2.11	Pure vs Side-Effect Functions (Engineering viewpoint)	5
2.12	Mutating vs Non-Mutating APIs (Important for Lists/Dicts)	6
3	Passing Collections: Mutation and Side Effects	6
3.1	In-place Mutation (Sometimes Useful)	6
3.2	Avoiding Unintended Mutation (Copying)	6
4	Recursion	6
4.1	What Is Recursion?	6
4.2	Example: Factorial	7
4.3	When to Avoid Recursion in Python	7
5	Functional Tools: <code>map</code>, <code>filter</code>, <code>lambda</code>	7
5.1	<code>map</code> : Transform Each Item	7
5.2	<code>filter</code> : Keep Items That Match a Condition	7
5.3	Pitfall: Iterators in Python 3	7
5.4	<code>lambda</code> : Small Anonymous Functions	7
5.5	Inner Functions and Closures	8
6	Pure Functions vs Side Effects	8

7 Demo Walkthrough: Cleaning Data with <code>map/filter</code>	8
7.1 What the Demo Is Teaching	8
7.2 Suggested Run Steps	9
8 Quiz and Solutions	9
8.1 Checkpoint 1 (Base Case)	9
8.2 Checkpoint 2 (Replace Loop with <code>map</code>)	9
8.3 Think-Pair-Share (Avoiding Recursion)	9
8.4 Exit Question (Mutation Visibility)	9
9 Practice Exercises (With Solutions)	9
9.1 Exercise 1: Pure vs In-place	9
9.2 Exercise 2: Recursive Sum	10
9.3 Exercise 3: Rewrite <code>map/filter</code> as a Comprehension	10
9.4 Exercise 4: Closure Practice	10
10 Further Reading	10

1 Lecture Overview

This lecture moves from basic function syntax to advanced usage patterns that appear in real codebases:

- passing collections safely (mutation vs copying),
- recursion (and how to avoid infinite recursion),
- functional tools: `map`, `filter`, and `lambda`,
- closures (functions defined inside functions), and
- pure functions vs side effects.

2 Types of Functions in Python (and When to Use Them)

In Python, a **function is an object**. That means you can store a function in a variable, pass it into another function, and return it from a function. This idea is the foundation of **functional patterns** such as `map/filter` and closures.

2.1 Built-in Functions (Provided by Python)

Python already includes many reliable, well-tested functions. Prefer built-ins when they solve the problem clearly and directly.

- **Numeric/aggregation:** `sum`, `min`, `max`
- **Sizes and membership:** `len`, `any`, `all`
- **Ordering:** `sorted` (returns a new list)
- **Type conversion:** `int`, `float`, `str`, `list`

Use case example: reporting a quick summary without writing a loop:

```
marks = [72, 81, 65, 90]
print(min(marks), max(marks), sum(marks) / len(marks))
```

2.2 User-Defined Functions (def)

You write a function when you want to **reuse logic**, **name an idea**, and **reduce repetition**. A good function usually does one job and has a clear input/output contract.

- **Prefer returning values** instead of printing inside the function.
- **Document side effects** (e.g., mutating a list argument).

```
def average(nums):
    return sum(nums) / len(nums)
```

2.3 Functions by Arguments and Return Value (Common academic view)

You may see functions classified as “types” based on whether they take input arguments and whether they return a value. This is a useful mental model. Important Python detail: **every function returns something**. If you do not write **return**, Python returns **None** automatically.

- **No arguments, no (useful) return:** often used for side effects (printing).
- **Arguments, no (useful) return:** often used to update something (mutating a list, writing a file).
- **No arguments, returns a value:** usually reads from elsewhere (global state, user input). Use carefully.
- **Arguments, returns a value:** most reusable style; can be pure and easy to test.

```
def greet():
    print("Hello") # returns None

def add_item(lst, x):
    lst.append(x) # returns None (mutates input)

def get_pi():
    return 3.14159

def area_circle(r):
    return 3.14159 * r * r
```

2.4 Function Parameter Styles (Practical Python)

Real Python code often uses different parameter styles to make APIs flexible and readable.

- **Positional arguments:** simplest and most common.
- **Keyword arguments:** improve readability and reduce ordering mistakes.

- **Default values:** make parameters optional (configuration-style).
- **Variable-length arguments:** `*args` (many positional) and `**kwargs` (many keyword).
- **Keyword-only arguments:** force clarity (parameters after `*` must be named).

```
def power(base, exp=2):
    return base ** exp

print(power(5)) # 25 (default exp=2)
print(power(base=2, exp=5)) # 32 (keyword arguments)

def mean(*nums):
    return sum(nums) / len(nums)

def clip(x, *, lo=0, hi=100):
    return max(lo, min(x, hi))
```

2.5 Anonymous Functions (lambda)

`lambda` creates a small function without a name. It is best when the function is **short**, **used once**, and the meaning is obvious from context.

- **Good use case:** key= in sorted, small transforms in map
- **Avoid when:** logic becomes complex (use `def` for readability)

```
words = ["pear", "banana", "fig"]
print(sorted(words, key=lambda w: len(w))) # key function
```

2.6 Higher-Order Functions (Functions that take/return functions)

A higher-order function either:

- takes a function as input (e.g., `map`, `filter`, `sorted(key=...)`), or
- returns a function as output (a function factory / closure).

Use case: customize behavior without rewriting the whole algorithm.

```
nums = [1, 2, 3, 4]
sq = list(map(lambda x: x * x, nums)) # transform
ev = list(filter(lambda x: x % 2 == 0, nums)) # select
```

Use case (returning a function): create a specialized function:

```
def make_multiplier(k):
    def mult(x):
        return x * k
    return mult
```

2.7 Predicate Functions (Return True/False)

Predicates are commonly used with `filter`, `any`, and `all`. Their job is to **decide**, not to compute a numeric result.

```
def is_valid_mark(x):
    return isinstance(x, int) and 0 <= x <= 100

marks = [95, None, 120, 70]
valid = list(filter(is_valid_mark, marks))
```

2.8 Key Functions (Used for Sorting/Ranking)

Key functions are used by `sorted`, `min`, and `max`. They define what “comparison” should mean in your problem.

```
students = [("A", 9.1), ("B", 8.3), ("C", 9.4)]
top = max(students, key=lambda t: t[1]) # compare by CGPA
```

2.9 Recursive Functions (Define a problem in terms of itself)

Recursion is a natural fit when a problem breaks into smaller subproblems of the same shape.

- **Good use cases:** tree traversal, nested structures, divide-and-conquer
- **Be careful:** recursion depth limit, call overhead in Python

For simple loops, prefer iteration (clearer + faster in Python).

2.10 Generator Functions (yield) (Lazy computation)

Generators produce values one at a time instead of building a full list in memory. **Use case:** large data, streaming, pipelines.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for x in count_up_to(3):
    print(x)
```

2.11 Pure vs Side-Effect Functions (Engineering viewpoint)

- **Pure:** no external changes; easy to test; good for reuse and composition
- **Side effects:** printing, file I/O, mutating arguments; necessary but should be controlled

In practice, aim for **pure core logic** and keep side effects at the edges (input/output layer).

2.12 Mutating vs Non-Mutating APIs (Important for Lists/Dicts)

Some operations **modify objects in place** (side effect), while others return a **new object**.

- `items.sort()` mutates the list
- `sorted(items)` returns a new list (original is unchanged)

This lecture's next section shows why this distinction matters when passing collections into functions.

3 Passing Collections: Mutation and Side Effects

Lists and dictionaries are mutable. When you pass them into a function, you pass *a reference to the same object*. That means the function can change the caller-visible object.

3.1 In-place Mutation (Sometimes Useful)

```
def add_item(lst, item):  
    lst.append(item)  
  
data = [1, 2]  
add_item(data, 99)  
print(data) # [1, 2, 99]
```

This can be efficient, but it is also a side effect. If a function mutates its inputs, it should be clearly documented (and ideally reflected in the name).

3.2 Avoiding Unintended Mutation (Copying)

If you want a function that behaves like “input in, output out” without changing input, copy first:

```
def sort_copy(items):  
    tmp = items.copy()  
    tmp.sort()  
    return tmp
```

This pattern is a stepping stone to writing **pure functions**.

4 Recursion

4.1 What Is Recursion?

A recursive function calls itself to solve a smaller version of the same problem. Two required parts:

- **Base case:** stops recursion.
- **Progress step:** reduces the problem size each call.

4.2 Example: Factorial

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)
```

If the base case is missing or never reached, Python raises a `RecursionError` after hitting the recursion depth limit.

4.3 When to Avoid Recursion in Python

Recursion is sometimes the clearest solution (trees, divide-and-conquer), but for simple loops Python iteration is usually:

- faster (less call overhead),
- safer (no recursion depth risk), and
- easier to debug.

5 Functional Tools: map, filter, lambda

5.1 map: Transform Each Item

`map(f, iterable)` applies `f` to each item.

```
nums = [1, 2, 3, 4]  
squares = list(map(lambda x: x * x, nums))
```

5.2 filter: Keep Items That Match a Condition

`filter(pred, iterable)` keeps items where `pred(item)` is true.

```
nums = [1, 2, 3, 4]  
evens = list(filter(lambda x: x % 2 == 0, nums))
```

5.3 Pitfall: Iterators in Python 3

In Python 3, `map` and `filter` return **iterators**, not lists. That is why we wrap them in `list(...)` when printing or reusing.

5.4 lambda: Small Anonymous Functions

`lambda` is useful for short functions used once, often as a **key** for sorting or as a mapping function. Example (sorting by length):

```
words = ["pear", "banana", "fig"]  
print(sorted(words, key=lambda w: len(w)))
```

If the lambda becomes complex, define a normal named function instead.

5.5 Inner Functions and Closures

An inner function can capture variables from the outer function.

```
def make_multiplier(k):
    def mult(x):
        return x * k
    return mult

triple = make_multiplier(3)
print(triple(10)) # 30
```

This is a closure: `mult` “remembers” `k`.

Encapsulation with Inner Functions

Inner functions are also useful for **encapsulation**: keeping helper logic private so it cannot be called incorrectly from outside.

```
def normalize_marks(marks):
    def clip(x):
        return max(0, min(x, 100))
    return [clip(m) for m in marks]
```

Here, `clip` is an internal helper used only inside `normalize_marks`. This keeps the public API small and focused.

6 Pure Functions vs Side Effects

Pure function:

- same input always gives same output, and
- no changes to external state (no mutation of input, no global writes).

Pure functions are easier to test and reason about. Side effects are sometimes necessary (writing a file, printing, updating a database), but should be kept controlled.

7 Demo Walkthrough: Cleaning Data with `map/filter`

Script: `demo/functional_tools.py`

7.1 What the Demo Is Teaching

- filter invalid values before processing,
- map a transformation across data,
- compare recursion vs iteration for summation.

7.2 Suggested Run Steps

```
python demo/functional_tools.py
```

Extension idea: rewrite the `map/filter` part using list comprehensions and compare which version reads better.

8 Quiz and Solutions

8.1 Checkpoint 1 (Base Case)

Question. Identify the base case in a recursive function. Why is it necessary?

Solution. The base case is the condition that stops recursion (e.g., `if n == 1: return 1` in factorial). Without it, the function would call itself forever (until Python raises `RecursionError`).

8.2 Checkpoint 2 (Replace Loop with map)

Question. Replace this loop with `map`:

```
nums = [1,2,3,4,5]
squares = []
for n in nums:
    squares.append(n*n)
```

Solution.

```
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x * x, nums))
```

8.3 Think-Pair-Share (Avoiding Recursion)

Prompt. When should you avoid recursion in Python? Provide one reason and one example.

Sample response. Avoid recursion for simple counting/accumulation loops because recursion is slower and can hit recursion depth limits. Example: summing a list is clearer and safer with a `for` loop than with recursion.

8.4 Exit Question (Mutation Visibility)

Question. If a function appends to a list passed as an argument, does the caller see the change? Why?

Solution. Yes. The function receives a reference to the same list object, so mutating it (`append/remove`) changes the object the caller holds.

9 Practice Exercises (With Solutions)

9.1 Exercise 1: Pure vs In-place

Task. Write two functions: one that sorts a list in place, and one that returns a sorted copy.

Solution.

```
def sort_in_place(items):
    items.sort()
    return items

def sorted_copy(items):
    return sorted(items)
```

9.2 Exercise 2: Recursive Sum

Task. Write a recursive function `sum_list(nums)`.

Solution.

```
def sum_list(nums):
    if not nums:
        return 0
    return nums[0] + sum_list(nums[1:])
```

Note: slicing creates new lists; for very large lists, an iterative approach is more efficient.

9.3 Exercise 3: Rewrite map/filter as a Comprehension

Task. Given `nums`, build a list of squares of even numbers.

Solution.

```
nums = [1, 2, 3, 4, 5]
even_squares = [n*n for n in nums if n % 2 == 0]
```

9.4 Exercise 4: Closure Practice

Task. Use `make_multiplier` to create a function that multiplies by 5.

Solution.

```
five_x = make_multiplier(5)
print(five_x(7)) # 35
```

10 Further Reading

- <https://docs.python.org/3/library/functions.html#map>
- <https://docs.python.org/3/library/functions.html#filter>

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Differentiate common function types and choose by use case
- Pass collections safely and reason about mutation
- Apply recursion with a correct base case
- Use `map`, `filter`, and `lambda`
- Recognize side effects in functions

Functions Are Objects (First-class)

- You can store a function in a variable
- You can pass a function into another function (*higher-order*)
- You can return a function from a function (closure / factory)

```
def square(x):  
    return x * x  
  
f = square  
print(f(5)) # 25
```

Function Types and Typical Use Cases

Type	When you typically use it
Built-in	fast + readable (<code>len</code> , <code>sum</code> , <code>sorted</code>)
User-defined (<code>def</code>)	reuse and name logic; easiest to test
Anonymous (<code>lambda</code>)	small one-off function for <code>key=</code> / <code>map</code>
Higher-order	take/return functions (customizable pipelines)
Generator (<code>yield</code>)	produce values lazily for large/streaming data
Recursive	natural fit for trees / nested structures (use carefully)

Predicate Functions (for filter)

- A predicate returns True/False
- Use it to keep only valid items

```
marks = [95, None, 120, 70]

def is_valid(x):
    return (
        isinstance(x, int)
        and 0 <= x <= 100
    )

valid = list(filter(is_valid, marks))
```

Key Functions (for sorted/min/max)

- A key function returns a value used for comparison
- Use it to sort/rank complex records (tuples, dicts, objects)

```
students = [
    ("A", 9.1),
    ("B", 8.3),
    ("C", 9.4),
]

top = max(students, key=lambda t: t[1])
```

Passing Collections

- Lists and dicts are mutable
- Functions can modify them in place
- Use copies when you want to preserve input

```
def add_item(lst, item):
    lst.append(item)
```

Worked Example: Avoid Unintended Mutation

```
def sort_copy(items):
    tmp = items.copy()
    tmp.sort()
    return tmp

data = [3, 1, 2]
print(sort_copy(data)) # [1, 2, 3]
print(data) # [3, 1, 2]
```

- Copy when you want a function without side effects

Recursion Basics

- A function calls itself
- Needs a base case to stop
- Each call reduces the problem size

Recursive Example

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)
```

Pitfall: Recursion in Python

- Missing base case leads to infinite recursion
- Deep recursion can hit recursion limits
- Prefer iteration for simple loops in Python

map and filter

- `map` applies a function to each item
- `filter` keeps items that satisfy a condition

```
nums = [1, 2, 3, 4]  
print(list(map(lambda x: x*x, nums)))  
print(list(filter(lambda x: x%2==0, nums)))
```

Pitfall: map/filter Return Iterators

- In Python 3, `map` and `filter` return iterators
- Wrap with `list(...)` to materialize results

Generator Functions (yield)

- Generates values one-by-one (lazy) instead of building a full list
- Saves memory and works well for large/streaming data
- Often used in pipelines and file processing

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
print(list(count_up_to(5)))
```

Lambda and Inner Functions

- `lambda` creates small anonymous functions
- Inner functions can capture variables (closures)
- Inner functions can hide helper logic (encapsulation)

```
def make_multiplier(k):  
    def mult(x):  
        return x * k  
    return mult
```

Encapsulation Example (Helper Inside a Function)

```
def normalize_marks(marks):  
    def clip(x):  
        return max(0, min(x, 100))  
    return [clip(m) for m in marks]
```

- `clip` is used internally and not exposed outside
- Keeps implementation details private and focused

Side Effects vs Pure Functions

- Pure functions do not change external state
- Side effects can make debugging harder
- Prefer pure functions when possible

Demo: Cleaning Data with `map/filter`

- Remove invalid values
- Transform items with `map`
- Compute recursive sum
- Extension: rewrite using comprehensions and compare readability

Script: `demo/functional_tools.py`

Checkpoint 1

Identify the base case in a recursive function. Why is it necessary?

Checkpoint 2

Replace this loop with `map`:

```
nums = [1,2,3,4,5]  
squares = []  
for n in nums:  
    squares.append(n*n)
```

Think-Pair-Share

When should you avoid recursion in Python? Provide one reason and one example.

Summary

- Passing collections can mutate inputs
- Recursion needs a base case
- `map/filter/lambda` support functional style
- Prefer pure functions for clarity and testing

Exit Question

If a function appends to a list passed as argument, does the caller see the change? Why?